Lab 5 : Scoping a Grounded Research Gap in LLM4APR

Topic : Beyond Search Scores : How File Localization Quality Affects AI Bug Fixing

Juan (东万) – L25020002

I. BACKGROUND & CURRENT AI CAPABILITIES

AI is now capable of fixing bugs in real-world software projects. In LLM-based automated program repair (LLM4APR), an AI agent reads bug reports, navigates and analyzes the project's codebase, and then writes a patch to resolve the issue. However, before an AI can fix a bug, it must first identify where the bug is located a step known as localization. Localization is the most challenging part of the entire process, consuming the majority of an agent's time and computational resources. Consequently, many tools have been developed specifically to assist AI in pinpointing the exact location of a bug. Some tools employ a hierarchical search moving from files to functions, and finally to specific lines of code while others trace the interconnections between different code segments, or provide concise summaries of the bug along with its location. Overall, these tools contribute to more effective repair outcomes.

II. THE UNRESOLVED GAP : FILE LOCALIZATION

In large-scale projects, localization tools can rarely pinpoint with absolute certainty which file contains a bug. Consequently, rather than providing the AI with a single file, these tools supply a list of candidate files those most strongly suspected of harboring the issue. The challenge, however, is that it remains unclear what constitutes an “ideal” list. If the list is too long, it will inevitably include many files unrelated to the bug; these extraneous files can distract the AI, clutter its limited context window, and potentially lead it to attempt fixes in the wrong files. Conversely, if the list is too short, the file actually containing the bug might be omitted, leaving the AI with no opportunity to address it.

III. ROOT CAUSE ANALYSIS

This gap persists due to three limitations in how localization has been studied to date:

1. **Overly simplistic testing setups**: Most studies only examine small bugs within a single function and compare just two scenarios. In these studies, the bug location is either provided perfectly or identified automatically. Furthermore, many repair systems assume the bug location is already known a condition rarely met in real-world projects, which typically consist of numerous files.
2. **Use of search scores as a proxy for repair outcomes**: Localization tools are primarily evaluated based on how effectively they assist AI in fixing bugs. However, a tool with a high search score does not necessarily produce or provide a better actual fix.
3. **Passing tests equated with success**: A fix is generally considered successful if it passes a project's test suite. Yet, these tests are not always sufficiently rigorous; consequently, a fix that passes the tests may still be incorrect, succeeding only by chance.

IV. PROPOSED SOLUTION

To address this gap, this study proposes a controlled experiment using a public benchmark containing real-world GitHub issues. The AI model and agent configuration remain constant, with the only variable being the list of files provided to the agent. In some experiments, irrelevant files are added to the set of relevant files to determine whether the AI becomes distracted. In others, certain files—which would otherwise be removed—are omitted to see if the AI can still successfully fix the bug. Bugs are also categorized by complexity, ranging from simple to complex. Furthermore, the study compares outcomes when the AI is provided only with the location of a bug versus when it is given both the location and a brief description of the issue. Each fix is subsequently evaluated using more rigorous testing to verify its correctness, rather than relying solely on whether it passed the initial tests. The findings will inform tool developers about which aspects of localization quality are truly significant, and will help users determine the optimal number of files to provide ensuring effective results without incurring unnecessary costs.

V. FORMULATED RESEARCH QUESTIONS (RQS)

RQ1. Precision versus recall

How do the precision and recall of the list of files provided to an AI repair agent affect the repair success rate and the associated costs?

Independent Variables: Precision and recall of the provided file list (manipulated by adding irrelevant files or removing correct ones)

Dependent Variables: Repair success rate (%) and token cost per bug

RQ2. Interaction with change complexity

Does the impact of localization quality on repair success depend on the complexity of the required fix?

Independent Variables: Localization condition (no localization, predicted localization, perfect localization), with repair complexity (single-file vs. multi-file) as a moderator

Dependent Variable: Repair success rate (%)

RQ3. Ground truth accuracy

Does providing the AI with a brief explanation of the bug, in addition to its location, lead to a higher number of correct repairs compared to providing only the location?

Independent Variable: Type of localization output (file path only; file and function; location plus an explanation)

Dependent Variables: Accuracy rate on more rigorous tests and the rate of overfitting (repairs that pass initial tests but fail more rigorous ones)